Day 41 – Adapters & OAuth 2.0 Token-Based Authentication

Class date: 27th August 2026 | Module: Security & Authentication in SAP CPI (Cloud Integration)

Correction made to your handwritten notes: You wrote the default OAuth token validity as **"6000 (1hr)"**. The correct default is **3600 seconds = 1 hour** (6000 seconds would actually be 1 hour 40 minutes). This has been fixed below. Everything else in your notes was conceptually correct – well done, especially on the 3-source explanation and the custom roles workflow.

1. What is Authentication? (start here first)

Authentication is the process of proving *"who is calling this system?"* before access is granted. Whenever one system (a sender) wants to talk to another system (a receiver) – for example, your CPI iFlow calling an SAP S/4HANA OData service, or an external system calling into your CPI iFlow – the receiving side must first verify the identity of the caller. In CPI, every adapter (HTTP, SOAP, OData, SFTP, IDoc, etc.) needs two things to make a call: the **URL** of the target system, and the **authentication details** that prove the caller is allowed to access it.

Example – HTTP Adapter flow: practice was done live on your BTP trial tenant. All iFlows built today were named starting with **Day_41**, as per your naming convention for hands-on practice iFlows.

2. Why Basic Authentication is not used in real projects

Until now, the authentication method used in practice was **Basic Authentication** – a static username and a static password sent with every call.

Question asked in class: "In real-time, do we use Basic Auth?" – Answer: **No.**

Reason: With Basic Auth, the username and password never change. If that combination is ever leaked, intercepted, or guessed, anyone can reuse it indefinitely to impersonate the system – there is nothing that automatically stops them. This is exactly like logging into Gmail once with a password and then that same password working forever with no expiry – it stays a permanent risk sitting out there.

Solution: For security, real-time projects use **OAuth**, which works on a **token-based approach**. Instead of a permanent password travelling with every call, a short-lived **access token** is generated first, and that token (not the raw credentials) is what gets sent with each request.

Corrected fact – token validity: The token is only valid for a limited time – by default **3600 seconds (1 hour)** on SAP BTP/Integration Suite. Once expired, it must be regenerated. This is what makes OAuth far more secure than Basic Auth: even if a token is stolen, it becomes useless after it expires.

3. The 3 sources required for OAuth authentication

To perform OAuth authentication, three pieces of information are required. Remember them as:

1. **Token URL** (also called Token Service URL) – the endpoint that issues the access token
2. **Client ID**
3. **Client Secret**

All three are obtained from the **Service Instance / Service Key** of the relevant service on **SAP BTP (Business Technology Platform) – Integration Suite runtime**. Concretely: go to the BTP Cockpit → the Integration Suite (or the specific API's own) Service Instance → create/view its Service Key → the JSON

returned there contains the **clientid**, **clientsecret**, and **token URL (tokenurl / uaa.url)**.

Flow: First, these 3 sources are used to generate the **OAuth access token**. Then that token is used for authentication when calling the main interface/receiver system.

4. How the token is passed – the Authorization header

Whenever authentication details are being passed to a receiver system, they are sent inside the request **header**, specifically the **Authorization** header, in one of these two formats:

```
Authorization: Bearer <token> ← used for OAuth tokens (this is the standard for token-based auth)

Authorization: Basic <token> ← used for Basic Authentication (base64-encoded user:password)
```

Correction: your notes wrote this as "Beaver" – the correct keyword is **Bearer**. "Bearer" means whoever *bears* (holds/carries) this token is treated as authenticated – the receiver does not check who the person is, only that they possess a valid, unexpired token.

5. Getting the token in practice – calling CPI via Postman

Now that we know how to *pass* the token, the next step is knowing how to *get* the token. Practice example: call the CPI system using Postman with OAuth credentials.

- Go to the API's respective **Service Instance** (or the Integration-related service instance) on BTP Cockpit.
- Inside it, all 3 resources are available: **Client ID, Client Secret, Token URL**.

Important – OAuth Grant Types: when using an OAuth token, there are multiple types of credential flows ("grant types"):

- **Client Credentials** grant – system-to-system calls using only Client ID + Client Secret
- **Username & Password** (Resource Owner Password) grant – Client ID + Client Secret + a specific username/password
- **Authorization Code** grant – involves a redirect/consent step, typically used for user-interactive login scenarios

All of the above are still **token-based authentication** at the core – the difference between them lies only in *how the token is generated* (what credentials are supplied to obtain it), not in how it is used afterward. Once generated, every flow uses the same Bearer-token header pattern shown above.

Additional facts noted in class:

- **Token type:** always **Bearer**.
- **Default validity: 3600 seconds (1 hour)** – corrected from the notes.
- Auto-renewal/updating of the authentication token in the **Security Material** is handled by SAP itself once the security artifact is deployed correctly – you do not manually refresh it for standard OAuth2 Client Credentials security material.

6. When other systems want to call our iFlow (as the receiver side)

So far the above covered CPI as the **caller** (sender role, calling out to another system). The reverse case is also common: another system wants to connect **into** our CPI iFlow. In that case, we (CPI team) need to

provide the external system with two things:

1. **Endpoint URL** – our iFlow's own endpoint (the sender HTTP/HTTPS adapter address)
2. **Service Instance details** – Client ID, Client Secret, Token URL (grant type: client_credentials)

This lets the external system generate its own token and call our iFlow the same way we call others – authentication works symmetrically in both directions.

7. Other authentication types beyond OAuth

OAuth is not the only authentication mechanism available. SAP CPI also supports **Certificate-based Authentication** (mutual TLS / client certificates), which is common for B2B/EDI-style integrations.

Authentication types worked on so far (useful for interview answers):

1. Basic Authentication
2. OAuth Authentication (Client Credentials)
3. Public Key Authentication (SFTP)
4. Certificate-based Authentication

From now onward, OAuth Authentication is the one to use as the default/standard for new integrations built in this course.

Interview tip (as explained in class): when asked "what authentication types have you worked with?", mention all types you've used, but always call out **OAuth** explicitly as the mandatory / most expected one, since it is the current industry standard for cloud-to-cloud and cloud-to-on-premise integration security.

8. Calling SAP Standard APIs

SAP publishes ready-made, pre-built standard APIs for common business processes, catalogued at **api.sap.com** (SAP Business Accelerator Hub). These can be called directly instead of building everything from scratch, and can also be consumed inside a custom iFlow using a **Request-Reply** pattern.

General URL pattern to call a standard API:

```
https://<host>/api/v1/<api-name>/<resource-or-message-processing-logs>
```

This same pattern can also be used to call your own custom iFlow's endpoint, since custom iFlows are exposed the same way as any other API once deployed.

9. Restricting iFlow access using Roles

Question: how is access to an iFlow controlled – i.e., who created the credentials that decide who can call which iFlow? Answer: **Based on the Role** assigned in the OAuth client / Service Instance.

Default role: ESBMessaging.send – this is the standard SAP-delivered role that grants permission to send messages into Cloud Integration iFlow endpoints. It is coarse-grained: it allows access to *all* deployed iFlows on the tenant.

Custom Roles are used when access needs to be restricted so that specific source systems can only call specific iFlows – not every iFlow on the tenant. Example scenario from class: System A should only be able to call iFlow1 and iFlow2 (via a "Source A Role"), while System B should only be able to call iFlow3 and iFlow4 (via a "Source B Role") – even though all four iFlows sit on the same tenant.

Step-by-step: setting up custom role-based restriction

1. **Step 1:** Create **Tenant-level Roles** in the CPI tenant – e.g. *iFlow1.Send*, *iFlow2.Send*. These roles are tied to the specific iFlow(s) they should permit access to.
2. **Step 2:** Create **separate Service Instances** in the BTP Cockpit, one per role/consumer, based on the service being used. Each Service Instance's Service Key returns its own Authorication URL/Token URL, Client ID, and Client Secret (Authentication details).
3. **Step 3:** After the iFlows (e.g. iFlow1 and iFlow2) are created and deployed, pair each one with its **separate Custom Role** – e.g. "Source A Role" and "Source B Role".
4. Add the Custom Roles under **Manage User Roles**, listing the tenant-level role names created in Step 1 (e.g. *iFlow1.Send*, *iFlow2.Send*) – only the names need to be added/mapped here.
5. Finally, create two different **Service Instances** and assign each the role it is permitted to use, so each consuming system only receives credentials scoped to the iFlow(s) it should be able to call.

Net effect: even though System A and System B both technically point at the same CPI tenant, the role assigned to each one's Service Instance credentials determines which specific iFlow endpoints they are authorized to reach – this is how iFlow access gets restricted on a vice-versa (cross) basis between multiple source systems and multiple iFlows.

Quick-recall summary

- Basic Auth = static, insecure for real-time use → OAuth = token-based, time-limited, secure.
- 3 sources needed for OAuth: **Token URL, Client ID, Client Secret.**
- Token is always type **Bearer**; default validity **3600 sec (1 hr)**.
- Grant types: Client Credentials / Username-Password / Authorization Code – all still token-based.
- Default access role for iFlows: **ESBMessaging.send**; use **Custom Roles** to restrict per-system, per-iFlow access.
- Auth types to know for interviews: Basic, OAuth, Certificate-based, SFTP Public Key – always highlight OAuth as the current standard.